

CELEBAL TECHNOLOGIES SUMMER INTERNSHIP 2026

WEEK 3 ASSIGNMENT

Advanced SQL using Subqueries, CTEs and Window Functions

Submitted By: Aditya Kumar, DIT University

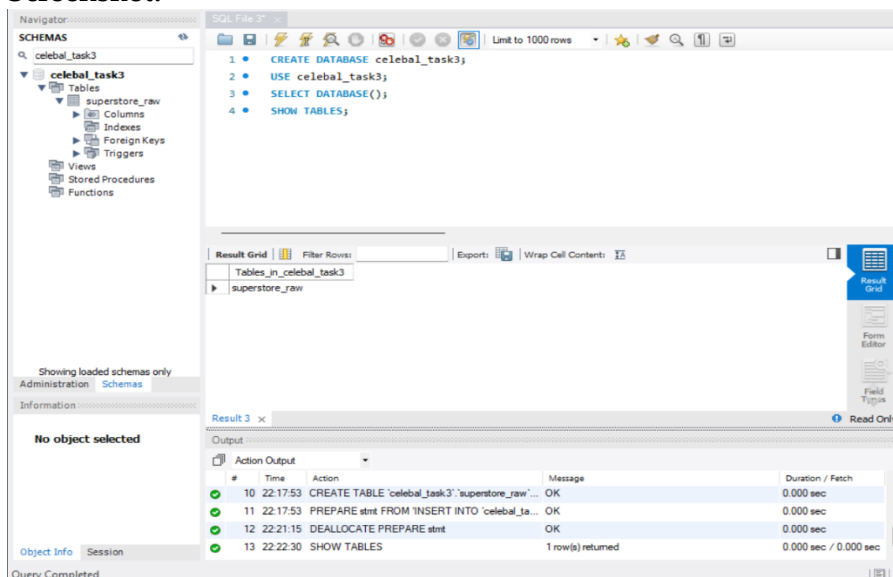
Objective

The objective of this assignment is to practice advanced SQL concepts including Subqueries, Common Table Expressions (CTEs), Window Functions, normalization of a raw dataset, and business-oriented analytical queries using the Superstore dataset.

1. Database Creation

Created a new MySQL database and verified the active schema.

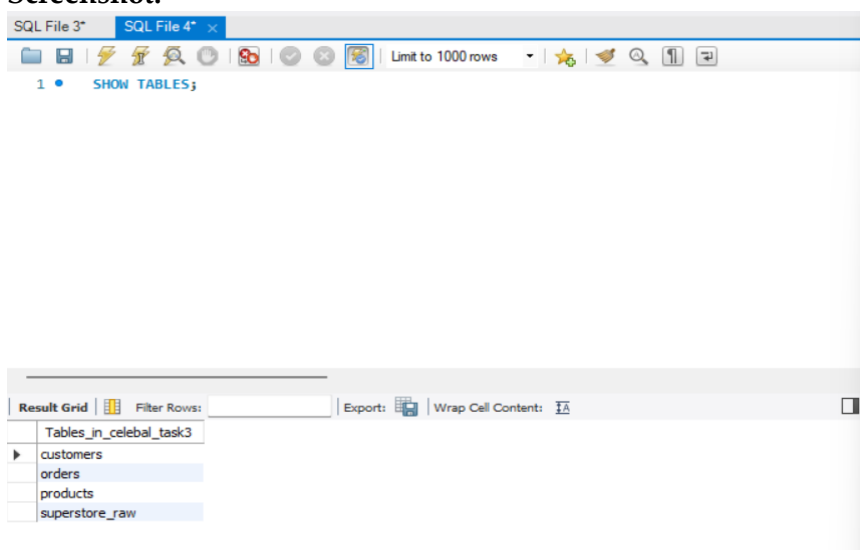
Screenshot:



2. Tables Created

The following tables were created for the assignment: superstore_raw, customers, orders, products

Screenshot:



3. Table Structures

Customers Table

The Customers table stores unique customer information extracted from the raw dataset.

Screenshot

The screenshot shows a SQL IDE interface with two tabs: 'SQL File 3*' and 'SQL File 4*'. The 'SQL File 4*' tab is active, displaying the following SQL code:

```
1 CREATE TABLE customers (  
2     customer_id VARCHAR(20) PRIMARY KEY,  
3     customer_name VARCHAR(100),  
4     segment VARCHAR(50),  
5     country VARCHAR(50),  
6     city VARCHAR(50),  
7     state VARCHAR(50),  
8     postal_code INT,  
9     region VARCHAR(50)  
10 );  
11 DESCRIBE customers;
```

Below the code editor, the 'Result Grid' shows the table structure:

Field	Type	Null	Key	Default	Extra
customer_id	varchar(20)	NO	PRI	NULL	
customer_name	varchar(100)	YES		NULL	
segment	varchar(50)	YES		NULL	
country	varchar(50)	YES		NULL	
city	varchar(50)	YES		NULL	
state	varchar(50)	YES		NULL	
postal_code	int	YES		NULL	
region	varchar(50)	YES		NULL	

The interface also includes a toolbar with icons for file operations, a 'Limit to 1000 rows' dropdown, and a 'Read Only' status indicator at the bottom right.

Orders Table

The Orders table stores unique order information and references the Customers table through a foreign key.

Screenshot

The screenshot shows a SQL IDE interface with two tabs: 'SQL File 3*' and 'SQL File 4*'. The 'SQL File 4*' tab is active, displaying the following SQL code:

```
1 CREATE TABLE orders (  
2     order_id VARCHAR(20) PRIMARY KEY,  
3     order_date VARCHAR(20),  
4     ship_date VARCHAR(20),  
5     ship_mode VARCHAR(50),  
6     customer_id VARCHAR(20),  
7     FOREIGN KEY (customer_id)  
8         REFERENCES customers(customer_id)  
9 );  
10 DESCRIBE orders;
```

Below the code editor, the 'Result Grid' shows the table structure:

Field	Type	Null	Key	Default	Extra
order_id	varchar(20)	NO	PRI	NULL	
order_date	varchar(20)	YES		NULL	
ship_date	varchar(20)	YES		NULL	
ship_mode	varchar(50)	YES		NULL	
customer_id	varchar(20)	YES	MUL	NULL	

The interface also includes a toolbar with icons for file operations, a 'Limit to 1000 rows' dropdown, and a 'Read Only' status indicator at the bottom right.

Products Table

The Products table stores unique product details including sales-related attributes.

Screenshot

The screenshot shows a SQL editor window titled 'SQL File 4' with the following SQL code:

```
1 CREATE TABLE products (  
2     product_id VARCHAR(30) PRIMARY KEY,  
3     category VARCHAR(50),  
4     sub_category VARCHAR(50),  
5     product_name VARCHAR(150),  
6     sales DOUBLE,  
7     quantity INT,  
8     discount DOUBLE,  
9     profit DOUBLE  
10 );  
11 DESCRIBE products;
```

Below the code, the 'Result Grid' displays the table structure:

Field	Type	Null	Key	Default	Extra
product_id	varchar(30)	NO	PRI	NULL	
category	varchar(50)	YES		NULL	
sub_category	varchar(50)	YES		NULL	
product_name	varchar(150)	YES		NULL	
sales	double	YES		NULL	
quantity	int	YES		NULL	
discount	double	YES		NULL	
profit	double	YES		NULL	

4. Data Population

Data was normalized from the raw Superstore dataset into separate Customers, Orders and Products tables.

Customers Population

Total Customers: 793

Screenshot

The screenshot shows a SQL editor window titled 'SQL File 4' with the following SQL code:

```
1 INSERT INTO customers (  
2     customer_id,  
3     customer_name,  
4     segment,  
5     country,  
6     city,  
7     state,  
8     postal_code,  
9     region  
10 )  
11 SELECT  
12     'Customer ID',  
13     MAX('Customer Name'),  
14     MAX(Segment),  
15     MAX(Country),  
16     MAX(City),  
17     MAX(State),  
18     MAX('Postal Code'),  
19     MAX(Region)  
20 FROM superstore raw
```

Below the code, the 'Result Grid' displays the table structure:

Field	Type	Null	Key	Default	Extra
customer_id	varchar(30)	NO	PRI	NULL	
customer_name	varchar(50)	YES		NULL	
segment	varchar(50)	YES		NULL	
country	varchar(50)	YES		NULL	
city	varchar(50)	YES		NULL	
state	varchar(50)	YES		NULL	
postal_code	varchar(50)	YES		NULL	
region	varchar(50)	YES		NULL	

Orders Population

Total Orders: 4931

Screenshot

The screenshot shows a SQL IDE with two tabs: 'SQL File 3*' and 'SQL File 4*'. The 'SQL File 4*' tab is active and contains the following SQL code:

```
1 • INSERT INTO orders (  
2     order_id,  
3     order_date,  
4     ship_date,  
5     ship_mode,  
6     customer_id  
7 )  
8 SELECT  
9     `Order ID`,  
10    MAX(`Order Date`),  
11    MAX(`Ship Date`),  
12    MAX(`Ship Mode`),  
13    MAX(`Customer ID`)  
14 FROM superstore_raw  
15 GROUP BY `Order ID`;  
16 • SELECT COUNT(*) AS total_orders  
17 FROM orders;
```

Below the code editor, the 'Result Grid' is visible, showing a single row with the column 'total_orders' and the value '4931'.

Products Population

Total Products: 1812

Screenshot

The screenshot shows a SQL IDE with two tabs: 'SQL File 3*' and 'SQL File 4*'. The 'SQL File 4*' tab is active and contains the following SQL code:

```
1 • INSERT INTO products (  
2     product_id,  
3     category,  
4     sub_category,  
5     product_name,  
6     sales,  
7     quantity,  
8     discount,  
9     profit  
10 )  
11 SELECT  
12     `Product ID`,  
13     MAX(Category),  
14     MAX(`Sub-Category`),  
15     MAX(`Product Name`),  
16     MAX(Sales),  
17     MAX(Quantity),  
18     MAX(Discount),  
19     MAX(Profit)  
20 FROM superstore_raw
```

Below the code editor, the 'Result Grid' is visible, showing a single row with the column 'total_products' and the value '1812'.

Customers

Verified that customer records were successfully inserted.

Screenshot

[illegible]

Orders

Verified that order records were successfully inserted.

Screenshot

16 • `SELECT * FROM orders LIMIT 5;`

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch

	order_id	order_date	ship_date	ship_mode	customer_id
▶	CA-2014-100006	9/7/2014	9/13/2014	Standard Class	DK-13375
	CA-2014-100090	7/8/2014	7/12/2014	Standard Class	EB-13705
	CA-2014-100293	3/14/2014	3/18/2014	Standard Class	NF-18475
	CA-2014-100363	4/8/2014	4/15/2014	Standard Class	JM-15655
	CA-2014-100391	5/25/2014	5/29/2014	Standard Class	BW-11065
✱	NULL	NULL	NULL	NULL	NULL

orders 8 × Apply Revert

Products

Verified that product records were successfully inserted.

Screenshot

[illegible]

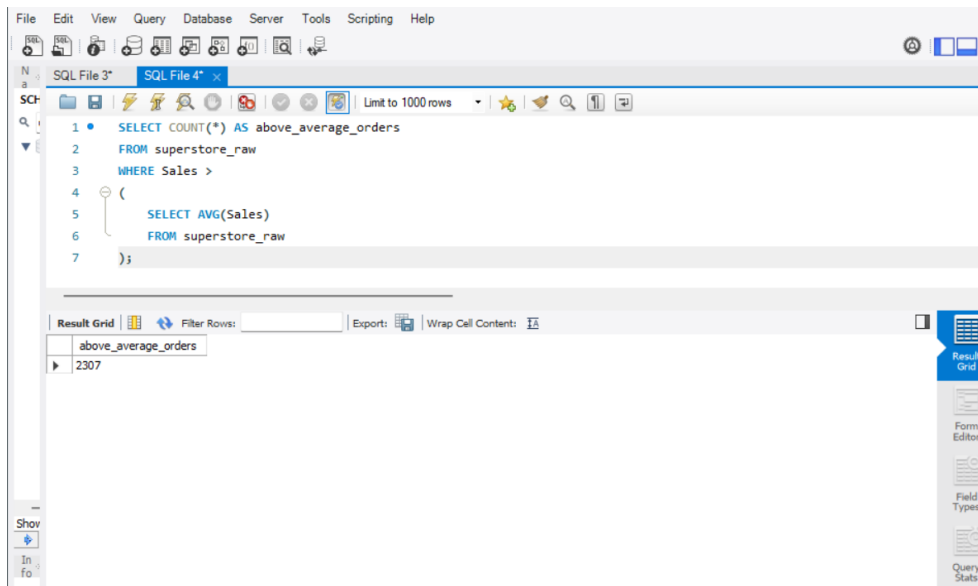
6. Assignment Questions

Question 1

Find customers whose total sales are greater than the average customer sales.

A Common Table Expression (CTE) was used to calculate customer sales, followed by a subquery to compare against the average sales.

Screenshot

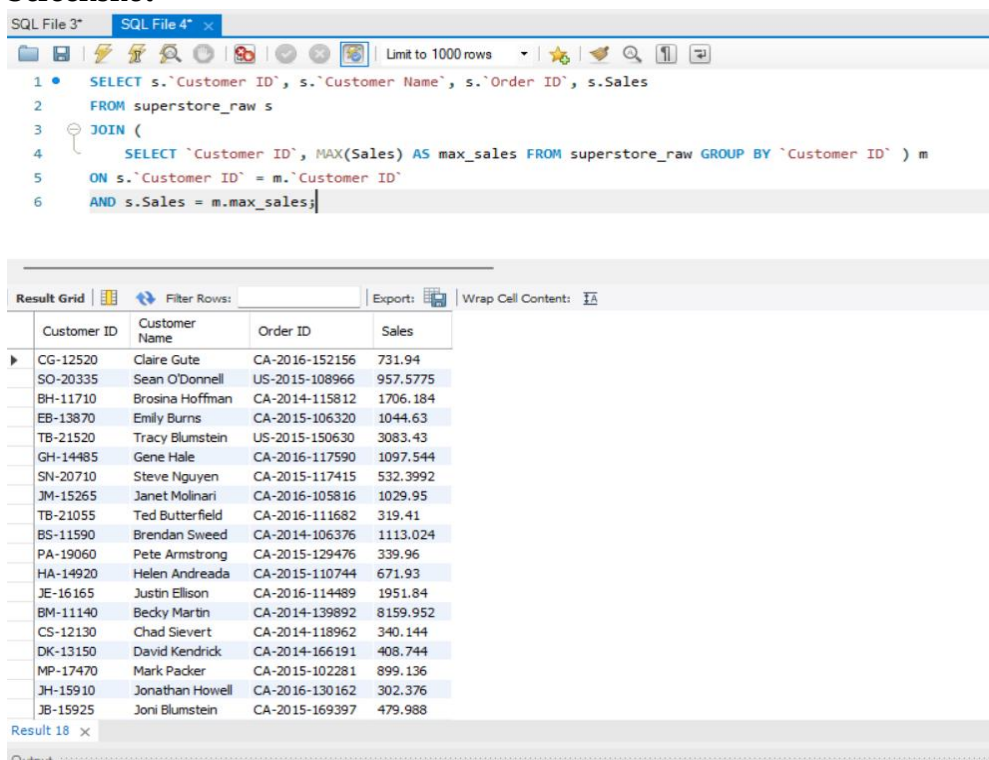


Question 2

Rank customers based on total sales.

The RANK() window function was used to assign rankings according to total sales.

Screenshot



Question 3

Assign row numbers for each customer's orders based on sales.

ROW_NUMBER() with PARTITION BY was used to number orders within each customer.

Screenshot

The screenshot shows a SQL query window with the following code:

```
1 WITH customer_sales AS (  
2     SELECT  
3         'Customer ID',  
4         'Customer Name',  
5         SUM(Sales) AS total_sales  
6     FROM superstore_raw  
7     GROUP BY  
8         'Customer ID',  
9         'Customer Name'  
10 )  
11 SELECT *  
12 FROM customer_sales  
13 ORDER BY total_sales DESC;
```

The result grid displays the following data:

Customer ID	Customer Name	total_sales
SM-20320	Sean Miller	25043.05
TC-20980	Tamara Chand	19017.847999999998
RB-19360	Raymond Buch	15117.339
TA-21385	Tom Ashbrook	14595.62
AB-10105	Adrian Barton	14355.610999999997
SC-20095	Sanjit Chand	14142.333999999999
KL-16645	Ken Lonsdale	14071.917
HL-15040	Hunter Lopez	12873.297999999999
SE-20110	Sanjit Engle	12209.438000000002
CC-12370	Christopher Conant	12129.072
TC-21120	Todd Sumall	11885.871

Question 4

Display the Top 3 customers based on sales.

A CTE and RANK() function were combined to retrieve the highest-performing customers.

Screenshot

The screenshot shows a SQL query window with the following code:

```
1 WITH customer_sales AS (  
2     SELECT  
3         'Customer ID',  
4         'Customer Name',  
5         ROUND(SUM(Sales), 2) AS total_sales  
6     FROM superstore_raw  
7     GROUP BY  
8         'Customer ID',  
9         'Customer Name'  
10 )  
11 SELECT * FROM customer_sales WHERE total_sales >  
12 ( SELECT AVG(total_sales) FROM customer_sales ) ORDER BY total_sales DESC;
```

The result grid displays the following data:

Customer ID	Customer Name	total_sales
SM-20320	Sean Miller	25043.05
TC-20980	Tamara Chand	19017.85
RB-19360	Raymond Buch	15117.34
TA-21385	Tom Ashbrook	14595.62
AB-10105	Adrian Barton	14355.61
SC-20095	Sanjit Chand	14142.33
KL-16645	Ken Lonsdale	14071.92
HL-15040	Hunter Lopez	12873.3
SE-20110	Sanjit Engle	12209.44
CC-12370	Christopher Conant	12129.07
TC-21120	Todd Sumall	11885.87

Question 5

Display customer rankings with total sales.

Customer sales were aggregated and ranked using the RANK() window function.

Screenshot

The screenshot shows a SQL query in SQL Developer. The query uses a CTE named 'customer_sales' to aggregate sales by customer, then uses RANK() to rank them by total sales in descending order.

```
1 WITH customer_sales AS (  
2     SELECT  
3         'Customer ID',  
4         'Customer Name',  
5         ROUND(SUM(Sales),2) AS total_sales  
6     FROM superstore_raw  
7     GROUP BY  
8         'Customer ID',  
9         'Customer Name'  
10 )  
11 SELECT 'Customer ID', 'Customer Name', total_sales, RANK() OVER ( ORDER BY total_sales DESC ) AS customer_rank  
12 FROM customer_sales;
```

The result grid shows the following data:

Customer ID	Customer Name	total_sales	customer_rank
SM-20320	Sean Miller	25043.05	1
TC-20980	Tamara Chand	19017.85	2
RB-19360	Raymond Buch	15117.34	3
TA-21385	Tom Ashbrook	14595.62	4
AB-10105	Adrian Barton	14355.61	5
SC-20095	Sanjit Chand	14142.33	6
KL-16645	Ken Lonsdale	14071.92	7
HL-15040	Hunter Lopez	12873.3	8
SE-20110	Sanjit Engle	12209.44	9
CC-12370	Christopher Conant	12129.07	10
TS-21370	Todd Sumrall	11885.87	11
GT-14710	Greg Tran	11820.12	12
BM-11140	Becky Martin	11609.9	13
SV-20365	Seth Vernon	11420.65	14

Question 6

Number customer orders using ROW_NUMBER().

ROW_NUMBER() was applied with PARTITION BY Customer ID to generate sequential numbering.

Screenshot

The screenshot shows a SQL query in SQL Developer. The query uses ROW_NUMBER() with PARTITION BY Customer ID to generate sequential numbering for each customer's orders.

```
1 SELECT  
2     'Customer ID',  
3     'Customer Name',  
4     'Order ID',  
5     Sales,  
6     ROW_NUMBER() OVER (  
7         PARTITION BY 'Customer ID'  
8         ORDER BY Sales DESC  
9     ) AS row_num  
10 FROM superstore_raw;
```

The result grid shows the following data:

Customer ID	Customer Name	Order ID	Sales	row_num
AA-10315	Alex Avila	CA-2016-103982	3930.072	1
AA-10315	Alex Avila	CA-2014-128055	673.568	2
AA-10315	Alex Avila	CA-2016-103982	431.976	3
AA-10315	Alex Avila	CA-2017-147039	362.94	4
AA-10315	Alex Avila	CA-2014-128055	52.98	5
AA-10315	Alex Avila	CA-2016-103982	41.72	6
AA-10315	Alex Avila	CA-2015-121391	26.96	7
AA-10315	Alex Avila	CA-2014-138100	14.94	8
AA-10315	Alex Avila	CA-2014-138100	14.56	9
AA-10315	Alex Avila	CA-2017-147039	11.54	10
AA-10315	Alex Avila	CA-2016-103982	2.304	11
AA-10375	Allen Arnold	CA-2016-131065	499.98	1
AA-10375	Allen Arnold	CA-2015-140921	149.97	2
AA-10375	Allen Arnold	CA-2017-100230	116.98	3

Question 7

Retrieve the Top 3 ranked customers.

A filtered CTE returned only the first three ranked customers.

Screenshot

The screenshot shows a SQL query in a text editor. The query uses a Common Table Expression (CTE) named `customer_sales` to calculate the total sales for each customer, ranked by total sales in descending order. The final query selects the top 3 ranked customers.

```
1 WITH customer_sales AS (  
2     SELECT  
3         'Customer ID',  
4         'Customer Name',  
5         ROUND(SUM(Sales),2) AS total_sales  
6     FROM superstore_raw  
7     GROUP BY  
8         'Customer ID',  
9         'Customer Name'  
10 ),  
11 ranked_customers AS (SELECT *, RANK() OVER ( ORDER BY total_sales DESC ) AS customer_rank FROM customer_sales )  
12 SELECT * FROM ranked_customers WHERE customer_rank <= 3;
```

The result grid shows the following data:

Customer ID	Customer Name	total_sales	customer_rank
SM-20320	Sean Miller	25043.05	1
TC-20980	Tamara Chand	19017.85	2
RB-19360	Raymond Buch	15117.34	3

Question 8

Join normalized customer data with sales rankings.

The Customers table was joined with the ranked sales data to display customer information along with rankings.

Screenshot

The screenshot shows a SQL query in a text editor. The query joins the `customer_sales` CTE with the `customers` table to display customer information along with their sales rankings.

```
1 WITH customer_sales AS (  
2     SELECT 'Customer ID', ROUND(SUM(Sales),2) AS total_sales  
3     FROM superstore_raw  
4     GROUP BY 'Customer ID'  
5 )  
6 SELECT c.customer_id, c.customer_name, cs.total_sales,  
7     RANK() OVER (  
8         ORDER BY cs.total_sales DESC  
9     ) AS customer_rank FROM customer_sales cs  
10 JOIN customers c ON cs.'Customer ID' = c.customer_id ORDER BY customer_rank;
```

The result grid shows the following data:

customer_id	customer_name	total_sales	customer_rank
SM-20320	Sean Miller	25043.05	1
TC-20980	Tamara Chand	19017.85	2
RB-19360	Raymond Buch	15117.34	3
TA-21385	Tom Ashbrook	14995.62	4
AB-10105	Adrian Barton	14355.61	5
SC-20095	Sanjit Chand	14142.33	6
KL-16645	Ken Lonsdale	14071.92	7
HL-15040	Hunter Lopez	12873.3	8
SE-20110	Sanjit Engle	12209.44	9
CC-12370	Christopher Conant	12129.07	10
TS-21570	Todd Sumrall	11885.87	11
GT-14710	Greg Tran	11820.12	12
BM-11140	Becky Martin	11609.9	13
SV-20365	Seth Vernon	11420.65	14
CJ-12010	Caroline Jumper	11079.74	15

7. Additional Business Analysis

Top 5 Customers by Sales

Identified the customers with the highest total sales.

Screenshot

The screenshot shows the MySQL Workbench interface with a query executed. The query selects the top 5 customers by total sales from the 'superstore_raw' table. The results are displayed in a table with columns: Customer ID, Customer Name, and total_sales.

```
1 • SELECT
2     `Customer ID`,
3     `Customer Name`,
4     ROUND(SUM(Sales),2) AS total_sales
5 FROM superstore_raw
6 GROUP BY
7     `Customer ID`,
8     `Customer Name`
9 ORDER BY total_sales DESC
10 LIMIT 5;
```

Customer ID	Customer Name	total_sales
SM-20320	Sean Miller	25043.05
TC-20980	Tamara Chand	19017.85
RB-19360	Raymond Buch	15117.34
TA-21385	Tom Ashbrook	14995.62
AB-10105	Adrian Barton	14355.61

Bottom 5 Customers by Sales

Displayed the customers with the lowest sales.

Screenshot

The screenshot shows the MySQL Workbench interface with a query executed. The query selects the bottom 5 customers by total sales from the 'superstore_raw' table. The results are displayed in a table with columns: Customer ID, Customer Name, and total_sales.

```
1 • SELECT
2     `Customer ID`,
3     `Customer Name`,
4     ROUND(SUM(Sales),2) AS total_sales
5 FROM superstore_raw
6 GROUP BY
7     `Customer ID`,
8     `Customer Name`
9 ORDER BY total_sales ASC
10 LIMIT 5;
```

Customer ID	Customer Name	total_sales
TS-21085	Thais Sissman	4.83
LD-16855	Lela Donovan	5.3
MG-18205	Mitch Gastineau	12.32
CJ-11875	Carl Jackson	16.52
RS-19870	Roy Skaria	22.33

Customers with Only One Order

Used GROUP BY and HAVING to identify customers who placed only one order.

Screenshot

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
1 SELECT
2     `Customer ID`,
3     `Customer Name`,
4     COUNT(DISTINCT `Order ID`) AS total_orders
5 FROM superstore_raw
6 GROUP BY
7     `Customer ID`,
8     `Customer Name`
9 HAVING COUNT(DISTINCT `Order ID`) = 1;
```

The results are displayed in the Result Grid below the query editor. The grid shows 27 rows of data, each representing a customer with a unique ID and name, and a total of 1 order.

Customer ID	Customer Name	total_orders
AO-10810	Anthony O'Donnell	1
AR-10570	Anemone Ratner	1
CJ-11875	Carl Jackson	1
JC-15385	Jenna Caffey	1
JR-15700	Jocasta Rupert	1
LD-16855	Lela Donovan	1
MG-18205	Mitch Gastineau	1
PH-18790	Patricia Hirasaki	1
RE-19405	Ricardo Emerson	1
RM-19750	Roland Murray	1
SM-20905	Susan MacKendrick	1
TC-21145	Theresa Coyne	1

Customers Above Average Sales

Used a CTE together with a subquery to identify customers whose total sales exceeded the average.

Screenshot

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
1 WITH customer_sales AS (
2     SELECT
3         `Customer ID`,
4         `Customer Name`,
5         ROUND(SUM(Sales),2) AS total_sales
6     FROM superstore_raw
7     GROUP BY
8         `Customer ID`,
9         `Customer Name`
10 )
11 SELECT * FROM customer_sales
12 WHERE total_sales > (SELECT AVG(total_sales) FROM customer_sales) ORDER BY total_sales DESC;
```

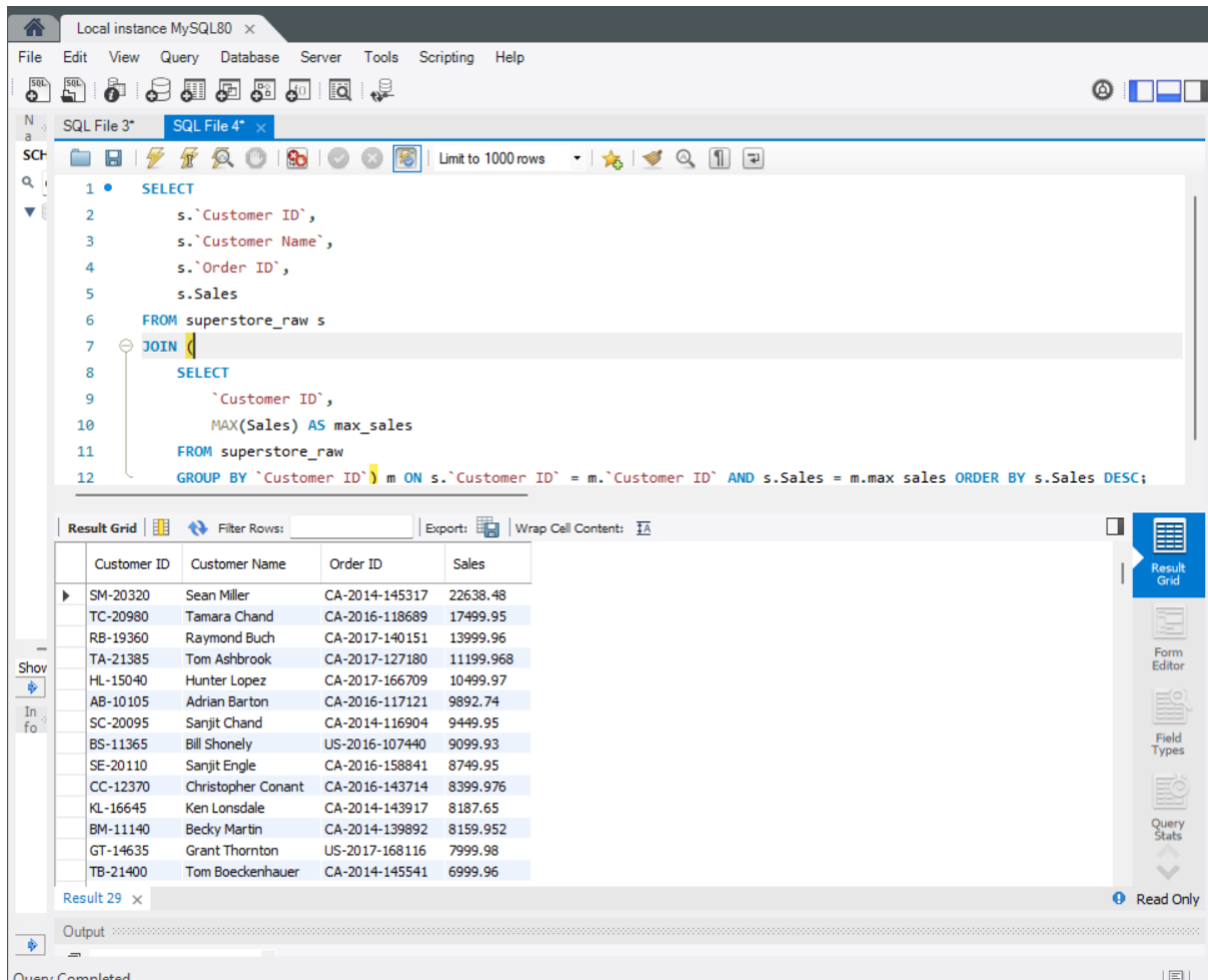
The results are displayed in the Result Grid below the query editor. The grid shows 28 rows of data, each representing a customer with a unique ID and name, and their total sales. The results are ordered by total sales in descending order.

Customer ID	Customer Name	total_sales
SM-20320	Sean Miller	25043.05
TC-20980	Tamara Chand	19017.85
RB-19360	Raymond Buch	15117.34
TA-21385	Tom Ashbrook	14595.62
AB-10105	Adrian Barton	14355.61
SC-20095	Sanjit Chand	14142.33
KL-16645	Ken Lonsdale	14071.92
HL-15040	Hunter Lopez	12873.3
SE-20110	Sanjit Engle	12209.44
CC-12370	Christopher Conant	12129.07
TS-21370	Todd Sumrall	11885.87
GT-14710	Greg Tran	11820.12
BM-11140	Becky Martin	11609.9
SV-20365	Seth Vernon	11420.65

Highest Value Order for Every Customer

Used a JOIN with a grouped subquery to retrieve each customer's highest-value order.

Screenshot



The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
1 SELECT
2     s.`Customer ID`,
3     s.`Customer Name`,
4     s.`Order ID`,
5     s.Sales
6 FROM superstore_raw s
7 JOIN
8     (SELECT
9         `Customer ID`,
10        MAX(Sales) AS max_sales
11     FROM superstore_raw
12     GROUP BY `Customer ID`) m ON s.`Customer ID` = m.`Customer ID` AND s.Sales = m.max_sales ORDER BY s.Sales DESC;
```

The Results tab displays the following data:

Customer ID	Customer Name	Order ID	Sales
SM-20320	Sean Miller	CA-2014-145317	22638.48
TC-20980	Tamara Chand	CA-2016-118689	17499.95
RB-19360	Raymond Buch	CA-2017-140151	13999.96
TA-21385	Tom Ashbrook	CA-2017-127180	11199.968
HL-15040	Hunter Lopez	CA-2017-166709	10499.97
AB-10105	Adrian Barton	CA-2016-117121	9892.74
SC-20095	Sanjit Chand	CA-2014-116904	9449.95
BS-11365	Bill Shonely	US-2016-107440	9099.93
SE-20110	Sanjit Engle	CA-2016-158841	8749.95
CC-12370	Christopher Conant	CA-2016-143714	8399.976
KL-16645	Ken Lonsdale	CA-2014-143917	8187.65
BM-11140	Becky Martin	CA-2014-139892	8159.952
GT-14635	Grant Thornton	US-2017-168116	7999.98
TB-21400	Tom Boeckenhauer	CA-2014-145541	6999.96

Conclusion

This assignment demonstrated the use of advanced SQL concepts including:

- Database normalization
- Subqueries
- Common Table Expressions (CTEs)
- Aggregate Functions
- Window Functions (RANK() and ROW_NUMBER())
- GROUP BY and HAVING
- JOIN operations
- Business-oriented analytical queries